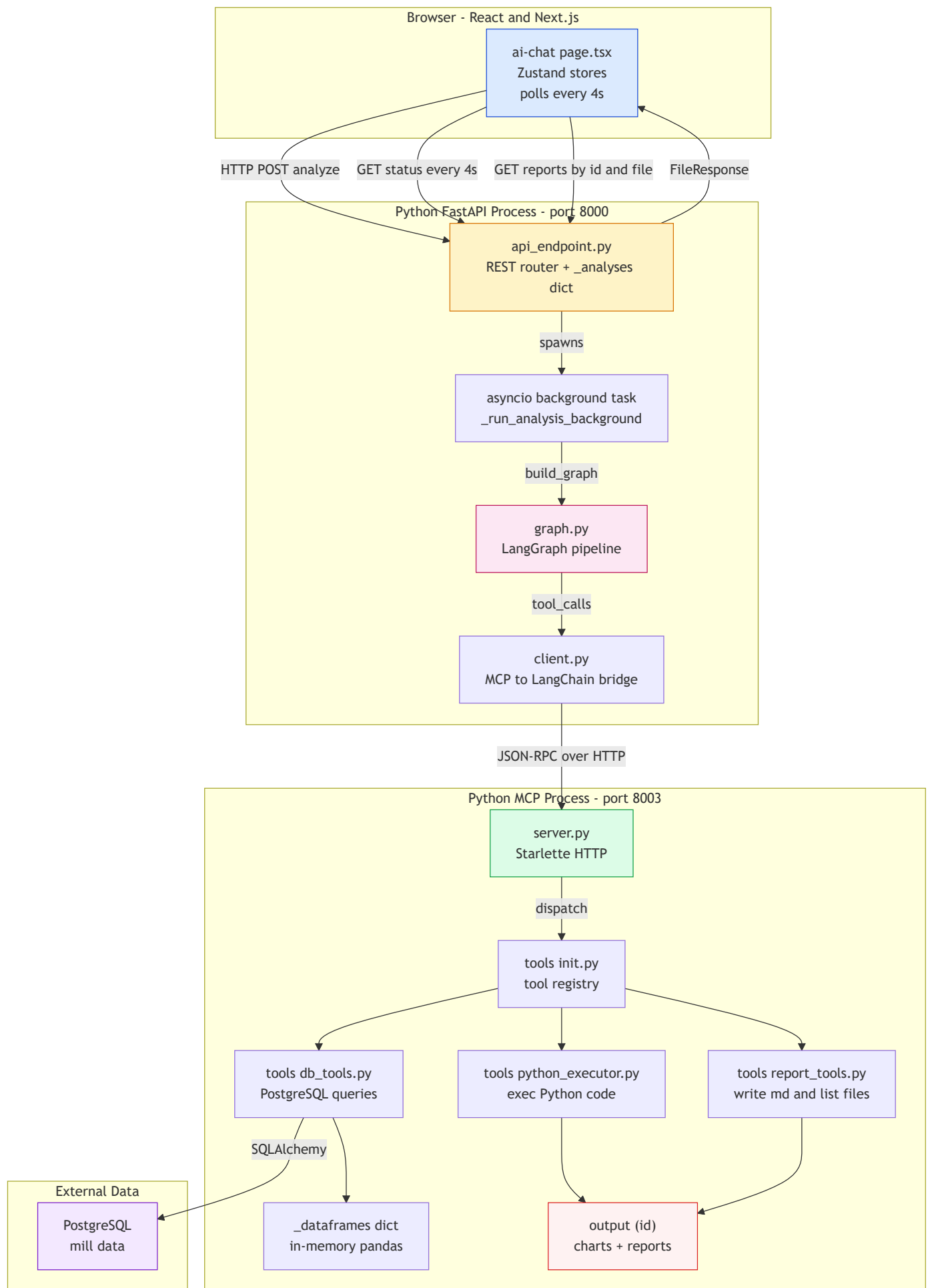


- 02 — Architecture (Beginner Edition)
 - One diagram, three boxes
 - Why two Python processes?
 - Data flow: a single request
 - Where does each piece of data live?
 - The three protocols
 - Why LangGraph instead of one big AI prompt?
 - Summary: one sentence per file

02 — Architecture (Beginner Edition)

Goal: You will know which process runs which code, where data lives, and why the system is split into two Python programs.

One diagram, three boxes



How to read it:

- **Browser** = the React app the user clicks and types in.

- **FastAPI Process** = the web server. It receives HTTP requests, spawns background tasks, and serves files back.
- **MCP Process** = the tool server. It never talks to the browser directly. It only talks to the FastAPI process via a private HTTP connection.
- **PostgreSQL** = the plant database with years of mill sensor data.

Why two Python processes?

Imagine a restaurant kitchen with two rooms:

Room	Job	Who works there
Front kitchen (FastAPI)	Takes orders, tells customers when food is ready, handles payments.	Waiters, cashier.
Back kitchen (MCP)	Actually cooks the food, stores ingredients, writes recipes.	Chefs, prep cooks.

The front kitchen does not know how to cook. It just passes notes to the back kitchen through a small window (the MCP connection).

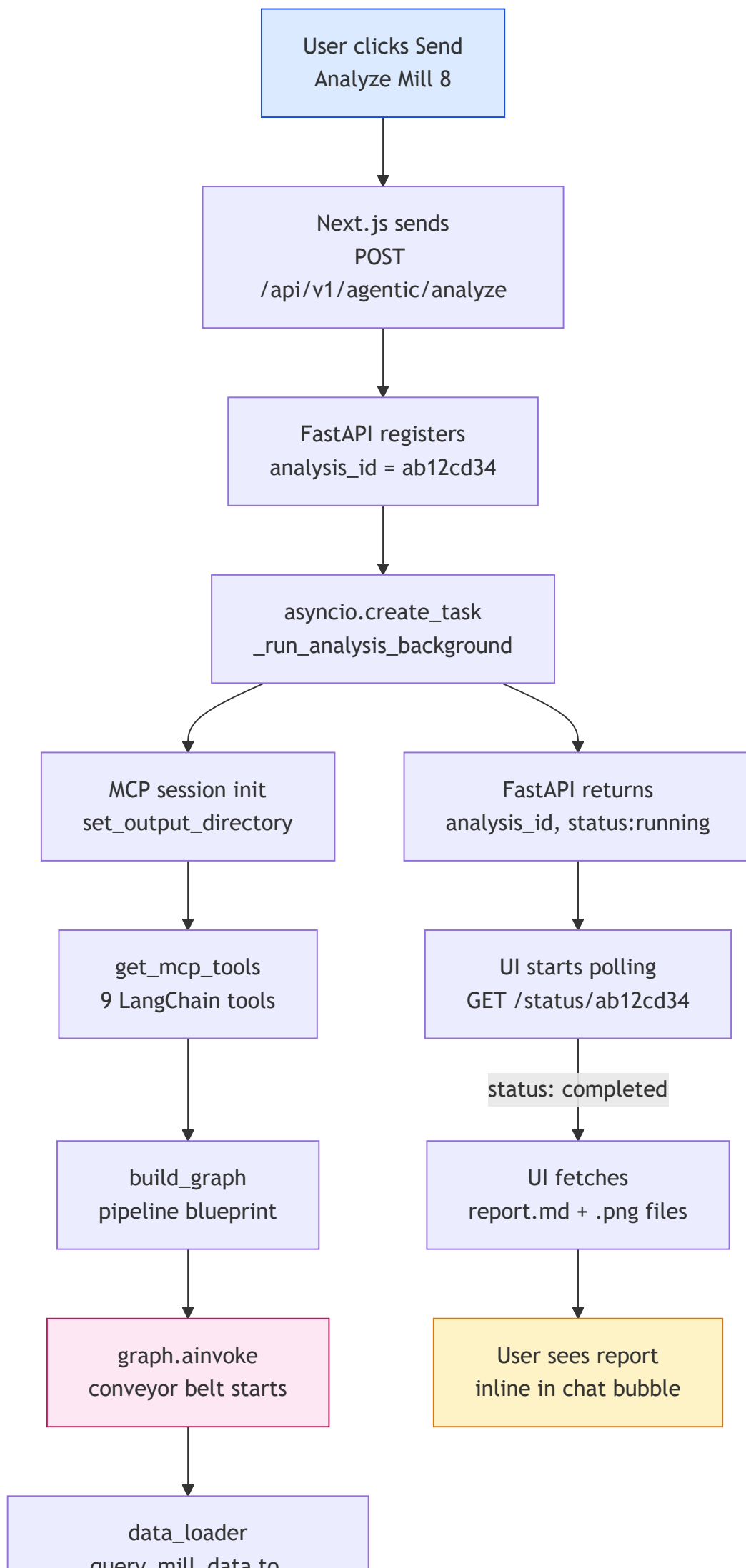
In our code:

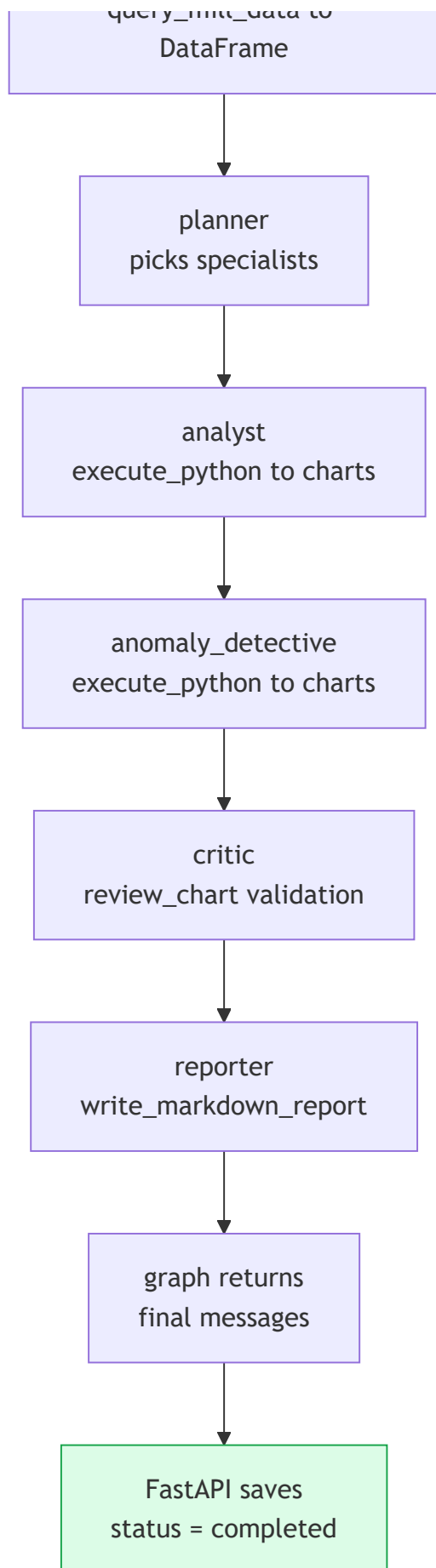
- **Front kitchen** = `api_endpoint.py` + `graph.py`. It decides what analysis to run and in what order.
- **Back kitchen** = `server.py` + `tools/*.py`. It loads data from PostgreSQL, runs Python code, and writes chart files.

Why not one big program?

1. **Memory isolation.** The MCP server loads huge pandas DataFrames. If it crashes, the FastAPI process stays alive and can tell the user "Something went wrong, please retry."
2. **Tool reuse.** The MCP server speaks a standard protocol (MCP). We could connect Claude Desktop, a CLI tool, or another LangChain app to the same server without writing new code.
3. **State management.** The MCP server is **stateful** (DataFrames stay in memory between tool calls). The FastAPI process is **stateless** (each request is independent, except for the `_analyses` cache).

Data flow: a single request





The key insight: The UI and the analysis run in parallel. The UI does not wait. It gets an ID immediately, then checks back every 4 seconds.

Where does each piece of data live?

Data	Lives in	Format	Who owns it	Survives restart?
Raw mill sensor rows	PostgreSQL	SQL tables	Database server	Yes
Loaded DataFrames	MCP process	pandas DataFrame (memory)	<code>tools/db_tools.py</code>	No
<code>_dataframes</code> dict	MCP process	Python dict of DataFrames	<code>tools/db_tools.py</code>	No
<code>_current_output_dir</code>	MCP process	string path	<code>tools/output_dir.py</code>	No
Analysis status + progress	FastAPI process	<code>_analyses</code> dict	<code>api_endpoint.py</code>	No
Analysis status + history	SQLite file	SQL tables	<code>analyses_db</code> (db.py)	Yes
Charts and reports	Disk	<code>.png</code> / <code>.md</code> files	<code>tools/report_tools.py</code>	Yes
Conversation list cache	Browser	localStorage JSON	<code>chat-store.ts</code>	Yes (per browser)

⚠ Important: Because `_dataframes` and `_current_output_dir` are global variables in the MCP process, running two analyses at the same time on the same MCP server would overwrite each other's data. The current design assumes one analysis at a time per MCP server.

The three protocols

The system uses three different communication styles:

Protocol	Used between	What it carries
HTTP REST	Browser ↔ FastAPI	JSON requests/responses, file downloads
MCP (JSON-RPC over HTTP)	FastAPI ↔ MCP server	Tool calls and results

Protocol	Used between	What it carries
SQL (SQLAlchemy)	MCP server ↔ PostgreSQL	Raw mill data queries

Why not just one protocol?

- REST is perfect for browser-to-server request/response.
- MCP is perfect for "I have a set of tools, please discover and run them."
- SQL is perfect for asking the database for 100,000 rows.

Each layer uses the tool best suited to its job.

Why LangGraph instead of one big AI prompt?

You might think: "Why not just send the user's question to Gemini and ask for a full report?"

Three reasons:

1. **Context window is too small.** A full analysis might generate 50,000 words of intermediate output. Gemini can only read ~20 messages at a time. LangGraph breaks the work into stages and only shows each specialist the relevant context.
2. **One specialist cannot do everything.** A forecaster needs time-series math. An anomaly detective needs isolation forests. A shift reporter needs shift schedules. Each specialist has a different system prompt and a different set of allowed tools.
3. **Quality control.** The manager review node acts like a senior engineer checking a junior's work. If the chart is wrong, the specialist reworks it before the reporter writes the final report.

Summary: one sentence per file

File	One-sentence job
<code>server.py</code>	Runs the tool server on port 8003.
<code>client.py</code>	Translates MCP tools into LangChain tools.

File	One-sentence job
<code>tools/__init__.py</code>	The registry: name → handler.
<code>tools/db_tools.py</code>	Fetches mill data from PostgreSQL into DataFrames.
<code>tools/python_executor.py</code>	Runs AI-generated Python code in a persistent namespace.
<code>tools/report_tools.py</code>	Writes <code>.md</code> reports and lists output files.
<code>graph.py</code>	Builds the multi-agent pipeline with dynamic routing.
<code>api_endpoint.py</code>	FastAPI router: HTTP in, background tasks out.
<code>page.tsx</code>	React chat page: sidebar, messages, input.
<code>chat-store.ts</code>	Zustand store: conversations, polling, message updates.

Next step: `03_mcp_deep_dive.md` to see the exact code inside `server.py` and `client.py`, or `07_request_lifecycle.md` to trace one real request from button-click to chart.